
UNIT 3 ENTITY RELATIONSHIP MODEL

- 3.0 Introduction
- 3.1 Objective
- 3.2 Entity Relationship (E-R) Model
 - 3.2.1 Entities
 - 3.2.2 Attributes
 - 3.2.3 Relationships
 - 3.2.4 E-R diagram Basics
 - 3.2.5 More about Relationships
 - 3.2.6 Extended E-R Features
 - 3.2.7 Defining Relationship for College Database
- 3.3 An Example
- 3.4 Conversion of E-R Diagram to Relational Database
- 3.5 Enhanced E-R Model
- 3.6 Converting E-R and EER Diagram to Relations
- 3.7 Summary
- 3.8 Solution/Answers

3.0 INTRODUCTION

In the previous unit of this block, you have gone through the concept of relational database management systems and one of the important languages for relational database – relational algebra. This unit explains, you about of an analysis model of the database system, known as Entity-Relationship (E-R) model. The E-R model is a widely used model for analysis of data requirements of an organisation. The E-R model is primarily a semantic model and is very useful in creating raw database design that can be further normalised. With the availability of object-oriented technologies, the E-R model has been extended to include object-oriented features. This unit also discusses these E-R extensions. We will also discuss the conversion of E-R diagrams to tables, in this unit.

3.1 OBJECTIVES

After going through this unit, you should be able to:

- Define and explain various components of E-R model;
- draw an E-R diagram for a given problem;
- convert an E-R diagram to a relational database;
- Explain the role of extended features of E-R model;
- Convert an EER diagram to relations.

3.2 ENTITY RELATIONSHIP (E-R) MODEL

Some of the important characteristics of E-R model are listed below:

- *Entity relationship model is a high-level conceptual data model.*
- *It allows you to describe the data involved in a real-world enterprise in terms of entities and their relationships.*
- *It is widely used to create an initial design of a database.*
- *It provides a set of useful concepts that make it convenient for a developer to move from a basic set of information to a detailed and precise description of information that can be easily implemented in a database system.*
- *It describes data as a collection of entities, relationships and attributes.*

In the following sections, we explain the basic terms used in the E-R model.

3.2.1 Entities

First let us answer the question: **What are entities?**

- An entity is an object of concern, which is used to represent the things in the real world, e.g., car, table, book, etc.
- An entity need not be a physical entity, it can also represent a concept in the real world, e.g., project, loan, etc.
- It represents a class of things, not any one instance, e.g., 'STUDENT' entity has instances of 'Ramesh' and 'Mohan'.

Entity Set or Entity Type: A collection of a similar kind of entity is called an Entity Set or entity type.

For the COLLEGE database, the objects of concern are Student, Faculty, Course and Department. The collections of all the students' entities form an entity set STUDENT. Similarly, collection of all the courses form an entity set COURSE.

You may please note that entity sets need not be disjoint. For example – an entity, say Mohan, may be part of the entity set STUDENT, the entity set FACULTY and the entity set PERSON.

Entity identifier - key attributes: An entity set usually has one or more attributes, which attains unique value for every distinct entity in a given entity set. Such an attribute or set of attributes is/are called key attribute(s) and its values can be used to identify each entity uniquely in the given entity set.

Strong entity set: An entity set which contains at least one key attribute is a Strong entity set. For example, a Student entity set would contain at least one key attribute *Enrolment number*, which is unique for every student, thus, the entity set Student is a strong entity set.

Weak entity set: Entity sets that do not contain any key attribute, and hence cannot be identified independently, are called weak entity sets. A weak entity cannot be identified uniquely by its attributes, therefore, are recognised in conjunction with the

primary key attributes of another strong entity on which its existence is dependent (called owner entity set).

Generally, a primary key of an owner entity set is attached to a weak entity set, which has identifying attributes, called discriminator attributes. These two together form the primary key of the weak entity set. The following restrictions must hold for the above:

- The owner entity set and the weak entity set must participate in one to many relationship set. This relationship set is called the identifying relationship set of the weak entity set.
- The weak entity set must have total participation in the identifying relationship.

One of the most common examples about the weak entity set is an entity set Dependent and the related Strong entity set Employee in an organisation. The Dependent entity set is used to list all the dependents of each employee of an organisation. The attributes of the Dependent entity set are: Dependent name, birth date, gender and relationship with the employee. Each Employee entity is said to own the dependent entities that are related to it. However, please note that the 'Dependent' entity does not exist of its own, it is dependent on the Employee entity. In other words, you can say that in case an employee leaves the organization, all dependents related to him/her also leave along with this employee. Thus, a 'Dependent' entity has no significance without the entity 'Employee'. Thus, it is a weak entity.

3.2.2 Attributes

Let us first define - **What is an attribute?**

An attribute is an element of an entity, which can contain a representative value. In other words, an entity is represented by a set of attributes.

For example, a Student entity set may consist of attributes - Roll no, student's name, age, address, course, etc. An entity will have a value for each of its attributes. For example, for a particular student, the following values can be assigned:

Roll No:	1234
Name:	Mohan
Age:	18
Address:	Z-894, Maidan Garhi, Delhi.
Course:	B.Sc. (H)

Domains: Each simple attribute of an entity type contains a possible set of values that can be attached to it. This is called the domain of an attribute. An attribute cannot contain a value outside this domain.

EXAMPLE- for STUDENT entity Age has a specific domain, integer values say from 15 to 90.

Types of attributes

Attributes attached to an entity can be of various types. They are explained below:

Simple: An attribute that cannot be further divided into smaller parts and represents the basic meaning is called a simple attribute. For example: Each of the attributes - 'FirstName', 'LastName', age of PERSON entity set are simple attributes.

Composite: Attributes that can be further divided into smaller units and each smaller unit contains specific meaning. For example, the attribute NAME of a FACULTY entity can be subdivided into First name, Last name and Middle name.

Single valued: Attributes having a single value for a particular entity. For Example, Age is a single valued attribute of a STUDENT entity.

Multivalued: Attributes that have more than one value for a particular entity is called a multivalued attribute. Different entities may have different numbers of values for these kinds of attributes. For multivalued attributes you must also specify the minimum and maximum number of values that can be attached. For example, the phone number for a PERSON entity is a multivalued attribute.

Stored and derived: Attributes that are directly stored in the database are called stored attributes. For example, 'Birth Date' attribute of a PERSON entity can be a stored attribute. However, there are certain attributes, whose value can be computed from the value of the stored attribute. For example, in the PERSON entity, the attribute 'Birth Date' can be used to compute the attribute Age of a person on a specific day. Thus, 'Birth Date' is a stored attribute, whereas Age may be a derived attribute for this entity.

3.2.3 Relationships

First, let us define the term relationships, i.e. **What Are Relationships?**

A relationship can be defined as:

- a connection or set of associations, or
- a rule for communication among entities:

Example: In a COLLEGE database, the association between student and course entity set, i.e., in the statement "Student **opts** Course" **opts** is an example of a relationship between the two entities Student and Course.

Relationship Sets

A relationship set is a set of relationships of the same type. For example, consider the relationship between two entity sets STUDENT and COURSE. Collection of all the instances of relationship **opts** forms a relationship set.

3.2.4 E-R Diagram Basics

The logical structure of a database is modeled using an E-R model, which is graphically represented with the help of an E-R diagram. The basic symbols using in an E-R diagrams are given in the following table:

Object	Represented by
Entity Set	Rectangle
Attribute	Ellipse
Relationship Set	Diamonds

Figure 3.1 shows different kinds of entities, attributes, relationships and participation constraints, which are explained in this Unit.

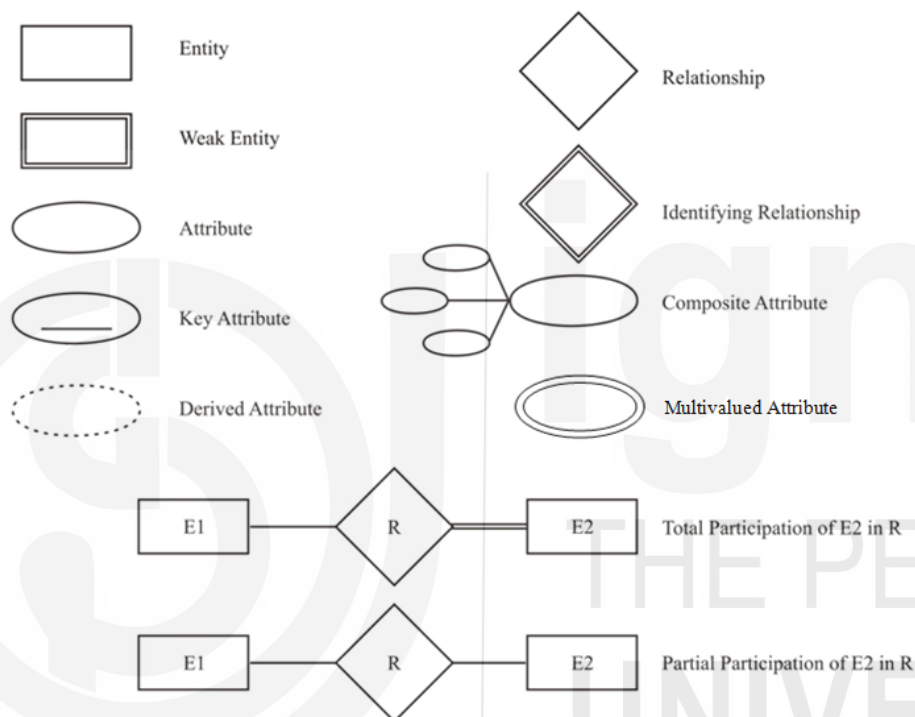


Figure 3.1: Symbols of E-R diagrams

3.2.5 More about Relationships

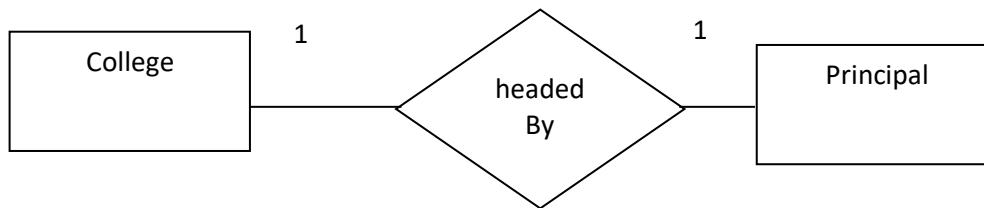
In this section, you will go through some of the important concepts, which are used for making good E-R models.

Degree of a relationship set: The degree of a relationship set is the number of participating Entity sets. The relationship between two entities is called a binary relationship. A relationship among three entities is called a ternary relationship. Similarly, a relationship among n entities is called an n -ary relationship.

Cardinality of a relationship set: Cardinality specifies the number of instances of an entity associated with another entity participating in a relationship. Based on the cardinality, binary relationships can be further classified into the following categories:

- **One-to-one:** An entity in A is associated with at most one entity in B, and an entity in B is associated with at most one entity in A.

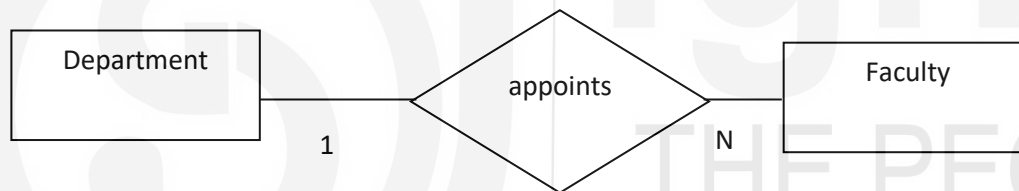
For example, the relationship *headedBy* between college entity set and principal entity set would be one-to-one, as one college can have at most one principal; and one principal can be principal of only one college. (This example assumes that a principal can be head of only one college.)



Similarly, you can define the relationship between University and Vice-Chancellor.

- *One-to-many*: Consider entity sets A and entity set B has a relationship cardinality A : B, as 1:N. This suggests that one specific entity of A may be related with several entities of entity set B.

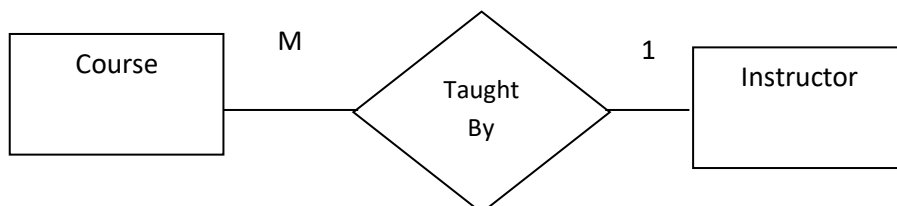
For example, relationship between Department entity set and Faculty entity set (assuming that a faculty member can work in only one department).



For example, in the diagram above, several faculty members may be appointed in one department, however, a specific faculty member will be appointed in precisely one department.

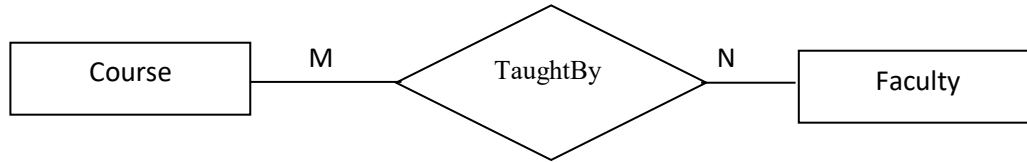
- *Many-to-one*: Consider entity sets A and entity set B has a relationship cardinality A : B, as N : 1. This suggests that several entities of A may be related with one specific entity of entity set B.

For example, the relationship between entity set Course and entity set Instructor. An instructor can teach various courses, but a single course can be taught only by one instructor. Please note this is an assumption.



Many-to-many: Entities in entity set A and entity set B are associated with any number of entities from each other.

For example, consider that a course can be taught jointly by many faculty members and each faculty member can teach several courses, then many-to-many relationship holds, as shown below:



Another example is shown in the diagram given below. The relationship cardinality $M : N$. This implies that an Author entity can be correlated to many Book entities, which are written by him/her. Further, a Book entity can also be correlated with several Author entities who have written the Book.



Recursive relationships: A recursive relationship is that relationships in which both the participating entity sets are the same entity set, however, the role of the entity set in each participation is different.

Participation constraints: The participation Constraints specify whether the existence of an entity depends on its being related to another entity via the relationship type. There are two types of participation constraints:

Total: When all the entities from an entity set participate in a relationship set, it is termed as the *total* participation. For example, the participation of the entity set Course in the relationship set 'TaughtBy' can be said to be *total* because every Course must be taught by a faculty.

Partial: When it is not necessary for all the entities from an entity set to participate in a relationship set, it is termed as *partial* participation. For example, the participation of the entity set Instructor in the relationship set 'TaughtBy' can be partial, since not every Instructor may be teaching a course.

3.2.6 Extended E-R Features

Although, the basic features of E-R diagrams are sufficient to design many database situations. However, with more complex relations and advanced database applications, it is required to use extended features of E-R models. The three such features are:

- Generalisation
- Specialisation, and
- Aggregation

We have explained them with the help of an example. More details on them are available in the further readings.

Example 1: A bank has an Account entity set. Any accounts of the bank can be one of two types: (1) Savings account and (2) Current account.

The statement above represents a specialisation/ generalisation hierarchy. It can be shown as:

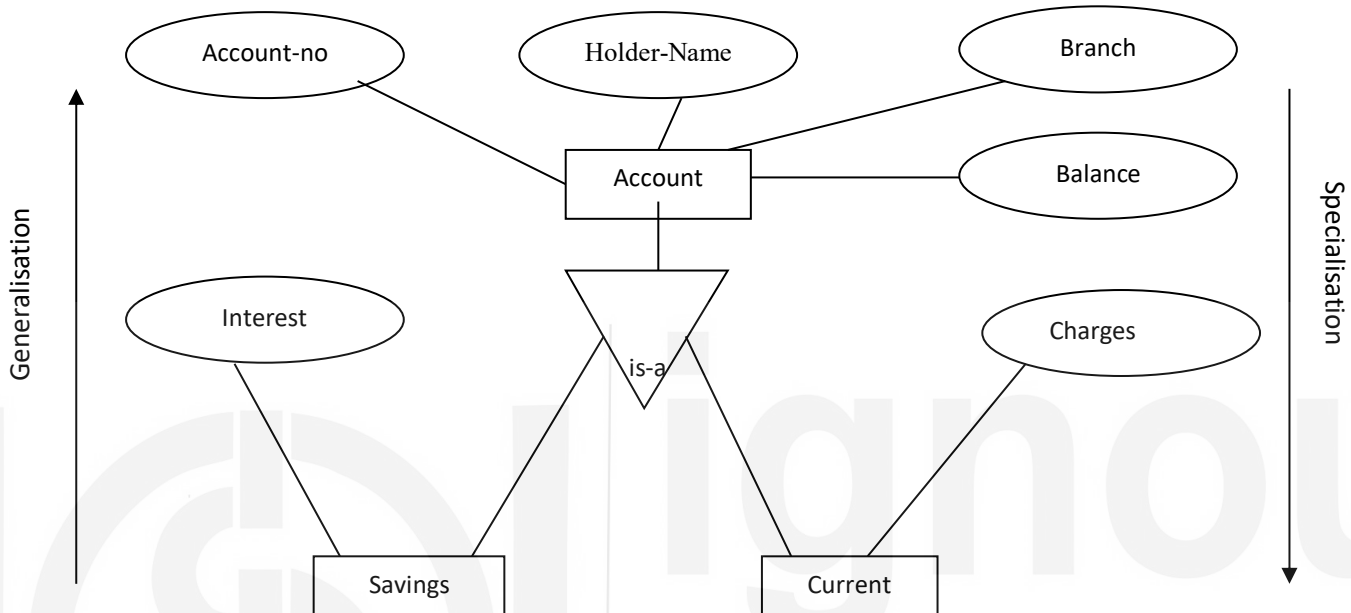


Figure 3.2: Generalisation and Specialisation hierarchy

Aggregation: One limitation of the E-R diagram is that they do not allow representation of relationships among relationships. In such a case the relationship along with its entities are promoted (aggregated to form an aggregate entity which can be used for expressing the required relationships). A detailed discussion on aggregation is beyond the scope of this unit you can refer to the further readings for more detail.

3.3 AN EXAMPLE

Let us explain it with the help of an example application. We will describe here an example database application of a COLLEGE database and use it for illustrating various E-R modeling concepts.

Problem Statement

A college database keeps track of Students, faculty, Departments and Courses. Following paragraphs gives the description of a COLLEGE database system.

A College contains various departments like Department of English, Department of Hindi, Department of Computer Science etc. Each department is assigned a unique

id and name. Some faculty members are appointed to each department and one of them works as head of the department.

There are various courses conducted by each department. Each course is assigned a unique id, name and duration.

The information contained for various objects are stated below:

- Faculty information contains name, address, department, basic salary etc. A faculty member is assigned to only one department but can teach courses of another department.
- Student's information contains Roll no (unique), Name, Address etc. A student can opt only for one course and one department only.
- Student's Parent or Guardian information to be recorded is - name of parent or guardian, age of the parent or guardian, gender and address of parent or guardian.

A student is allowed to take only one course. A student is assisted by his/her guardian. A faculty works in a department, which is headed by a faculty member. A course is taught by a faculty, who is allowed to teach several courses.

Defining Entities and Attributes

One of the simplest ways to identify the entities is to look for the proper nouns. This gives us possible set of entities in the problem statement are:

Student, Faculty, Department, Course, Guardian

The attributes of these entities can be found from the statements. You may also note except of Guardian:

Student (Rollno – Primary Key, Name, Address)

Faculty (Id – Primary Key, Name, Address, Basic_Sal)

Department (D_No, - Primary Key, D_Name)

Course (Course_ID – primary key, Course_Name, Duration)

Guardian (Name, Address, Relationship)

Please note that the Guardian is a weak entity. It is related to the strong entity Student.

Defining Relationship

Using the concepts defined earlier, we have identified that strong entities in COLLEGE database are Student, Faculty, Course and Department. This database also has one weak entity called Guardian. You can specify the following relationships among these entities. Further, these relationships also show the relationship cardinality and participation constraints:

1. **Head_of** is a 1:1 relationship between Faculty and Department. Participation of the entity Faculty is *partial* since not all the faculty members participate in this relationship (as all cannot be the head), while the participation from the Department side is *total* since every department has one head. Please also note that this relationship has an attribute Date_from, which stores the date from which the given appointment was applicable.

2. **Works_in** is a 1:N relationship between Department And Faculty, as one department can have many faculty, whereas a faculty is associated with only one department. Participation from both the sides is total, as every department has at least one faculty and every faculty must belong to a department.
3. **Opts** is a 1:N relationship between Course and Student. Participation from the Student side is *total* because we are assuming that each student opts for one course. But the participation from the course side is *partial*, since there can be courses that have no students.
4. **Taught_by** is a M: N relationship between Faculty and Course, as a Faculty can teach many courses and a Course can be taught by many faculty members.
5. **Enrolled** is a 1:N relationship between Student and Department as a student is allowed to enroll for only one department at a time. A student must enroll in a Department. However, a newly created Department may not have a student.
6. **Assisted_by** is a 1:N relationship between Student and Guardian as a student can have more than one local guardian and one local guardian is assumed to be related to one student only. The weak entity Guardian has *total* participation in the relation “Assisted_by”.

Now, you are ready to make an E-R diagram for the college database. The E-R diagram.

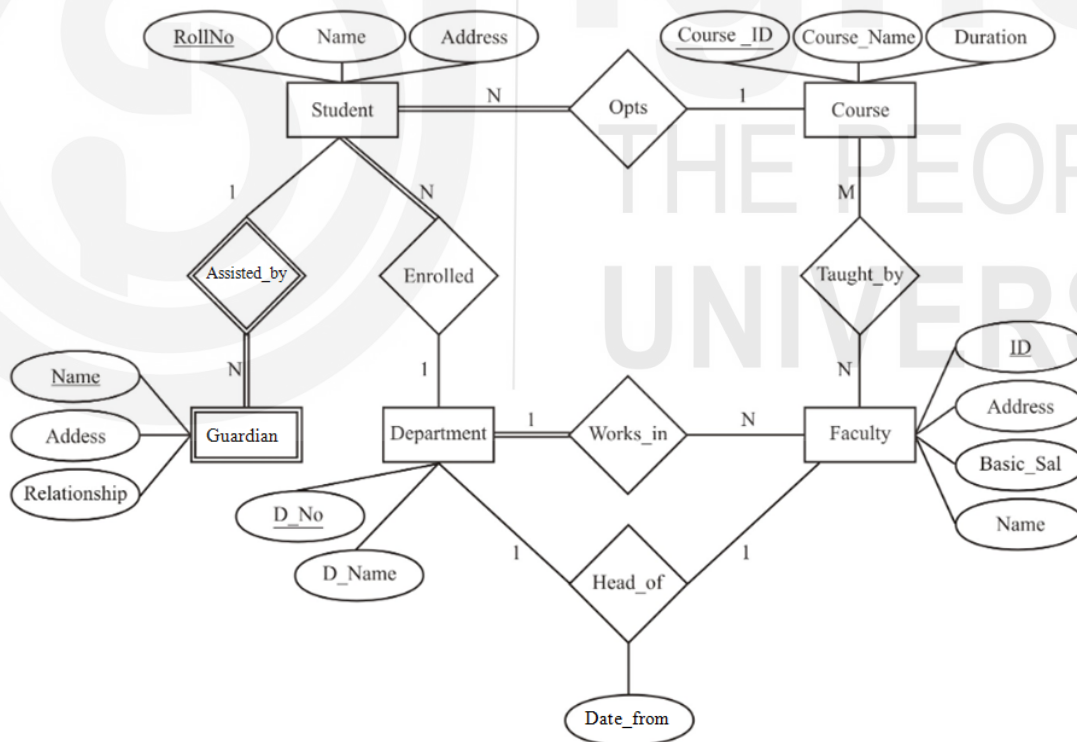


Figure 3.3: E-R diagram of COLLEGE database

3.4 CONVERSION OF E-R DIAGRAM TO RELATIONAL DATABASE

A relational database management system is designed from an E-R diagram, which represents various entities and relationships among entities for an application using the following methods.

Conversion of entity sets:

Strong entity set: For each strong entity set E in the E-R diagram, you create a relation R containing all the simple attributes of E. The primary key of the relation R will be one of the key attributes of R.

For example, the entities Student, Faculty, Course and Department, which are strong entities, relations as shown in Figure 3.4 would be created.

Student (Rollno, Name, Address)
Faculty (Id, Name, Address, Basic_Sal)
Department (D_No, D_Name)
Course (Course_ID, Course_Name, Duration)

Figure 3.4: Conversion of Strong Entities to relations

II) For each weak entity type W in the E-R Diagram, you create another relation R that contains all simple attributes of W. Further, you add the key attribute(s) of the owner entity set (say KP) of W in R. The primary key to this relation R is – <KP + Discriminator attribute of W> and foreign key is KP, which references the owner entity of W.

For example, conversion of weak entity Guardian into relation is shown in Figure 3.5. Please note that the owner entity of the Guardian entity is the strong entity Student, whose key is RollNo. Therefore, the key to Guardian relation is RollNo+Name. The Foreign key in Guardian relation is RollNo, which refers to Student relation.

Guardian (RollNO, Name, Address, Relationship)
Foreign Key: RollNO refers to relation Student

Figure 3.5: Conversion of weak entity Guardian to relation.

Conversion of relationship sets

Binary Relationships

I) One-to-one relationships:

For each 1:1 relationship set in the E-R diagram involving two entities E1 and E2 you choose one of the two entities (say E1), preferably the one with total participation, and add the primary key attribute of the other entity E2, in the relation of entity E1. Make this added attribute in E1 relation as a foreign key attribute to the relation

created from another entity (E2). You should also include all the simple attributes of the relationship type, if any, in the relation E1.

For example, the Head_of relationship in Figure 3.3 is 1:1. The two entities participating in the relationship are - Department and Faculty. The participation of the Department entity is *total* in the Head_of relationship. Therefore, the ID attribute, which is the primary key of the Faculty entity is added to Department relation. Please note, you can rename this attribute in the Department relation, if needed. In addition, this attribute in Department relation will be the foreign key to the Faculty relation. Further, the attribute Date_from of the Head_of Relationship is also added to the Department relation. This is shown in Figure 3.6. Please note that you will keep information in this the relation only stores the ID of the current head and Date from which s/he is the head. Please also note that you will not create a separate table of the relationship Head_of.

Department (D_No, D_Name, Head_ID, Date_from))

The ID attribute of Faculty relations is added to Department relation and has been renamed as Head_ID.

Foreign Key: Head_ID references ID attribute in Faculty relation

Figure 3.6: Converting 1:1 relationship (No new relation is added)

II) One-to-many or many-to-one relationships:

Both relationship sets involve two entity sets, say E1 and E2. Further, assume that E1 is on the many side and E2 is on the one side of the relationship set. You just need to include the primary key of the relation of entity on the one side (E2 in the case as above) to the relation created for the entity set of many side (E1 in the present case). You should include all simple attributes (or simple components of a composite attributes of relationship set, if any) in the relation of E1. Please note that now the relation of E1 has a foreign key reference to the relation of E2.

For example, the Works_in relationship between the entities Department and Faculty. For this relationship, the entity at N side is Faculty, therefore add primary key attribute of entity Department, i.e., D_No as a foreign key attribute in relation created for Faculty entity set. This is shown in Figure 3.7

Faculty (Id, Name, Address, Basic_Sal, D_No))

The Works_in relationship has been included in Faculty relation.

D_No is a foreign key and references the relation Department.

Figure 3.7: Converting 1:N relationship to relation (No new table is added in this case)

III) Many-to-many (M : N) relationship:

Consider that a M : N relationship set is binary with two participating entities, say Entity1 and Entity2. For this type of relationship set a relation is created. This relationship set should contain the Primary key of Entity1 (say PKE1), as well as the Primary key of Entity2 (say PKE2). In addition, any attribute of the relationship set is added to the relation. The primary key of this newly formed relation of the M : N relationship set is the composite primary key PKE1+PKE2. Please also note that for this new relation of the relationship set, there exists two foreign keys – PKE1, which refers to the relation of Entity1 and PKE2, which refers to the relation of Entity set Entity2.

For example, the $m : n$ relationship Taught_by between entity sets Course and Faculty should be represented as a new table. The structure of the table will include the primary key of Course and primary key of Faculty entities. Please note that both Course and Faculty relations remain unchanged.

Add the following relation into already existing list of relations:

Taught_by (Course_ID, ID)

Primary Key: Course_ID + ID

Foreign Keys:

Course_ID references Course relation

ID references Faculty relation

This relation has no other attribute, as the relationship has no attribute.

Figure 3.8: Converting $m : n$ relationship Taught_by into relations

N-ary Relationships

There are several cases for creating relations for n-ary relationships. A very general case is presented here. For each n-ary relationship set R where $n > 2$, you create a new table S to represent R. You should include the primary key of all the participating entity sets as the foreign key attributes in S. You should include any simple attributes of the n-ary relationship set (or simple components of complete attributes) as attributes of S. The primary key of S is usually a combination of all the foreign keys that reference the relations representing the participating entity sets. Figure 3.8 is a special case of n-ary relationship, i.e. a binary relationship.

Multivalued attributes:

For each multivalued attribute 'A' of an entity set E, you can create a new relation R that includes an attribute corresponding to the primary key attribute of the relation entity E that represents the entity set or relationship set that has as an attribute. The primary key of R is then a combination of A and the primary key of relation of E. For example, if a Student entity had RollNo, Name and PhoneNumber attributes, where phone number is a multivalued attribute, then you will create two tables for this entity as given below:

Student (RollNo, Name)

Phone (RollNo, PhoneNumber)

Converting Generalisation / Specialisation hierarchy to tables:

A simple rule for conversation may be to decompose all the specialised entities into relations in case they are disjoint. For example, for the E-R diagram of Figure 3.1, you can create the two tables as:

Saving_account (account-no, holder-name, branch, balance, interest).

Current_account (account-no, holder-name, branch, balance, charges).

The other way might be to create tables that are overlapping (not disjoint) for example, assuming that in the E-R diagram of Figure 3.2 contains overlapping subclasses, then you would be creating the following three relations:

The first relation would be for the for higher level entity:

account (account-no, holder-name, branch, balance)

The specialisation entities will contain the Primary key of the generalised entity and all the attributes of entity itself, as shown below:

saving (account-no, interest)

current (account-no, charges)

Thus, the information about a single account can be found in all the three relations.

Check Your Progress 1

- 1) A company wants to develop an application to store information about its clients. Find the possible entities and relationships among the entities. Show the step-by-step procedure to make an E-R diagram for the application.

.....

.....

.....

.....

- 2) An employee works for a department. If the employee is a manager, then s/he manages the department. Every employee works on at least one or more projects, which are controlled by various departments of a company. An employee can have many dependents. Draw an E-R diagram for the above company. Find all possible entities and their relationships.

.....

.....

.....

.....

- 3) A supplier, located in only one-city, supplies various parts for the projects of different companies located in various cities. You can name this database as “supplier-and-parts”. Draw the E-R diagram for the supplier-and-parts.

.....

.....

.....

.....

- 4) Convert the E-R diagram created for question 2 above into a relational database.

.....

.....

3.5 ENHANCED E-R MODEL

Enhanced E-R models can help in designing of relational and object-relational database systems. In addition, to E-R modeling concepts, the Enhanced ER model includes:

- 1) Subclass and Super class
- 2) Inheritance
- 3) Specialisation and Generalisation.

As discussed earlier, an entity may be an object with physical existence or it may be an object with a conceptual existence. An entity is depicted by a set of attributes. Sometimes, an entity can consist of explicit sub-groupings. For example, an entity *vehicle* consists of sub-groups – Truck (or Commercial Vehicles), Light Motor Vehicles (or Car), Two-Wheelers (or Scooter), etc. Every sub-grouping must belong to entity set *vehicle*, therefore, these sub-groupings are called a subclass of the *vehicle* entity set and the *vehicle* itself is called the super class for each of these sub-classes.

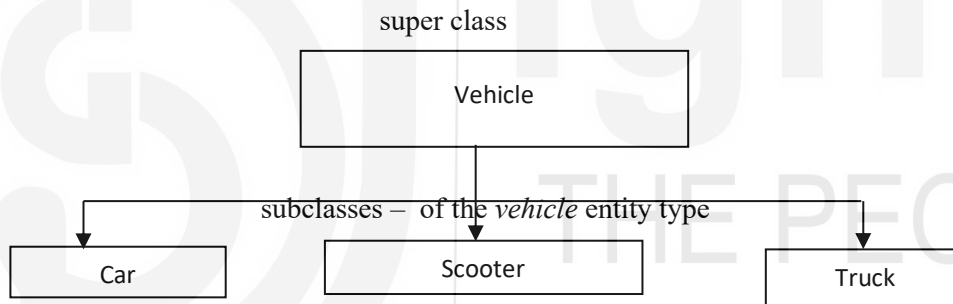


Figure 3.9: A class hierarchy

The relationship between a super class and any of its subclasses is called class/subclass relationship. It is often called an IS-A relationship because of the way we refer to the concept, as you would write, “Car *is-a* vehicle”. The member entity of the sub-class represents the same real world as the member entity of the super class. If an entity is a member of a sub-class, by default it must also become a member of the super class; whereas it is not necessary that every entity of the super class must be a member of its sub-class. An entity that is a member of a sub-class inherits all the attributes of its super class. An entity set is identified by its attributes and the relationship sets in which it participates; therefore, a sub-class entity also inherits all the relationships in which the super class participates. According to **inheritance** the sub-class inherits attributes and relationships of the super class. In addition, every subclass can contain its own attributes and relationships in which it has participated.

Specialisation is a process in which an entity set (super class) is modeled as a set of sub-entity sets (sub-classes) by using a specific distinguishing characteristic of the

super class. For example, in Figure 3.9, the super class *vehicle* is modeled using sub classes - Truck (or Commercial vehicle), Car (or Light Motor Vehicle), Scooter (or Two-wheelers)) using the *Type* attribute of the *vehicle* class. Please note that several specialisations hierarchies can be modeled using a super class by using different distinguishing characteristics. Figure 3.10 shows how you can represent a specialisation with the help of an EER diagram.

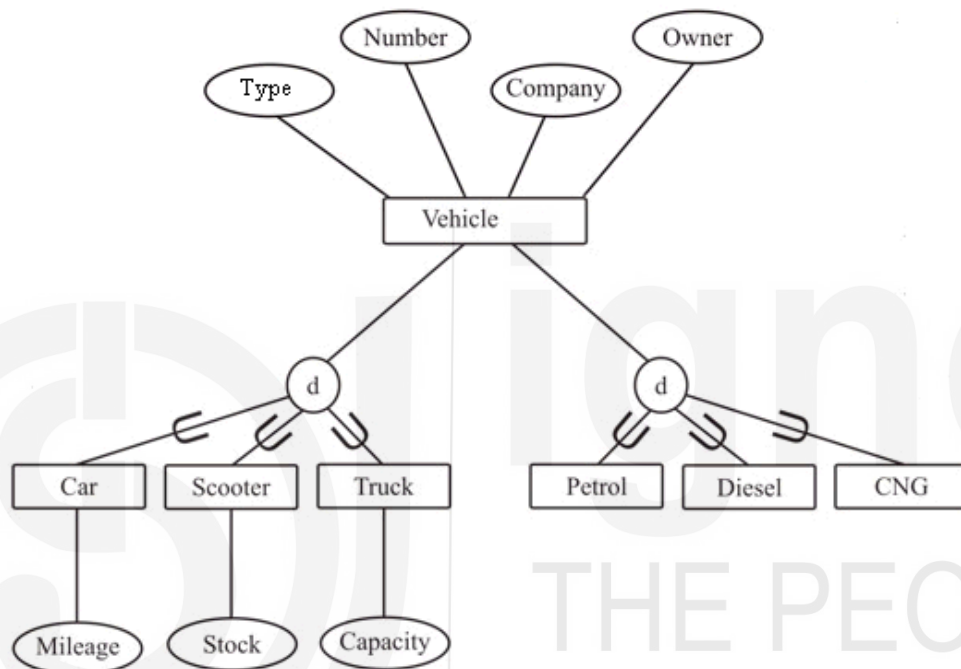


Figure 3.10: EER diagram showing more than one specialization from one super class

In Figure 3.10, letter 'd' in the circle indicates that all these subclasses are **disjoint** in nature, i.e. all the vehicle entities are disjoint, as they can be part of only one of the subclass. Please also notice that in Figure 3.10, common attributes, like vehicle number, owner name etc., are attributes of the super class, whereas attributes like mileage of car, stock of scooter and capacity of truck are the attributes of the sub-classes. Please note that an entity will be appearing twice in the EER diagram – once in the sub-class and the other in the super class (Please refer to Figure 3.11).

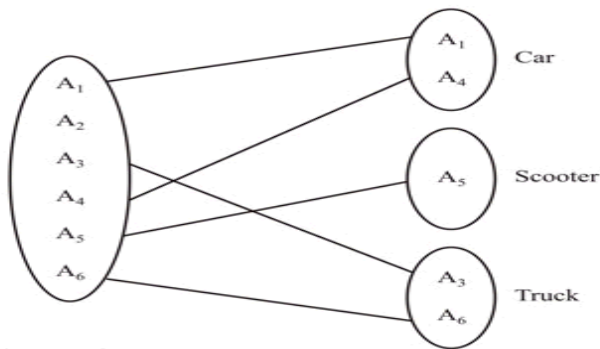


Figure 3.11: Sharing of members of the super class vehicle and its sub-classes

Hence, the specialisation is a set of sub-classes of an entity set, which establishes additional specific attributes with each sub-class and establishes additional specific relationship sets between a sub-class and other entity types or other sub-classes.

Generalisation is the reverse process of specialisation; in other words, it is a process of representing entity sets consisting of entities, which have almost similar attributes excepting few. The similar attributes of these entity set form the super class. For example, the entity set Car and Truck can be generalised into entity set Vehicle. Therefore, Car and Truck can now be sub-classes of the super class generalised class Vehicle (Refer to Figure 3.12).

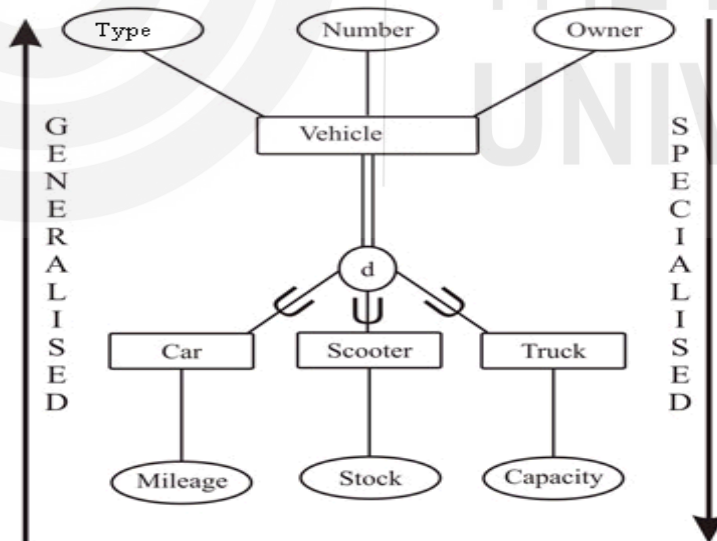


Figure 3.12: Generalisation and Specialisation

Constraints and Characteristics of Specialisation and Generalisation: A super class may either have a single sub-class or many sub-classes in specialisation. In

case of only one sub-class you do not use circle notation to show the relationship of sub-class/super class. Sometimes in specialisation, the subclass becomes the member of the super class after satisfying a condition on the value of some attributes of the super class. Such sub-classes are called *condition-defined* sub-classes or predicate defined subclasses. For example, the Vehicle entity set has an attribute vehicle “Type”, as shown in Figure 3.12.

You can specify the condition of membership for a subclass – car, truck, scooter – by the predicate – vehicle ‘Type’ of the super class vehicle. Therefore, a vehicle object can be a member of the sub-class, if it satisfies the membership condition for that sub-class. For example, to be a member of sub-class Car a vehicle entity must have the condition vehicle “type = car” as true. A specialisation in which an attribute or a set of attributes of the super class (called the *defining attribute* of specialisation) specify membership condition of the sub-classes, is termed as *attribute-defined* specialisation. In case no membership condition is stated for specialisation, a database user determines the membership. Such specialisation is termed as user-defined specialisation.

Disjointness is also the constraints to a specialisation, which specifies that a given entity cannot be a member of more than one sub-classes for a specific specialisation hierarchy. For example, in Figure 3.12, the symbol ‘d’ in circle stands for disjointness, as an entity can be a member of a single sub-class only. But if the real-world entity is not a disjoint entity sets of the sub-classes may overlap. This is represented by an (o) in the circle. For example, if you classify a class Book into sub-classes Textbooks and Reference Books, then you may like to define a specific book in both the sub-classes. This is a case of Overlapping constraints.

When every entity in the super class must be a member of some subclass in the specialisation it is called total specialisation. But if every entity does not necessarily need to belong to any of the subclasses, it is called partial specialisation. The total is represented by a double line. This is to note that in specialisation and generalisation the deletion of an entity from a super class implies automatic deletion from sub-classes belonging to the same; similarly, insertion of an entity in a super class results in insertion of the entity in all the sub-classes for which the attributes of this entity fulfills the constraints of attribute-defined specialisation. In case of total specialisation, insertion of an entity in a super class implies compulsory insertion in at least one of the sub-classes of the specialisation.

Union: In some cases, a single class has a similar relationship with more than one class. For example, the sub class ‘Car’ may be owned by two different types of owners: Individual or Organisation. Both these types of owners are different classes; thus, such a situation can be modeled with the help of a Union (Refer to Figure 3.13).

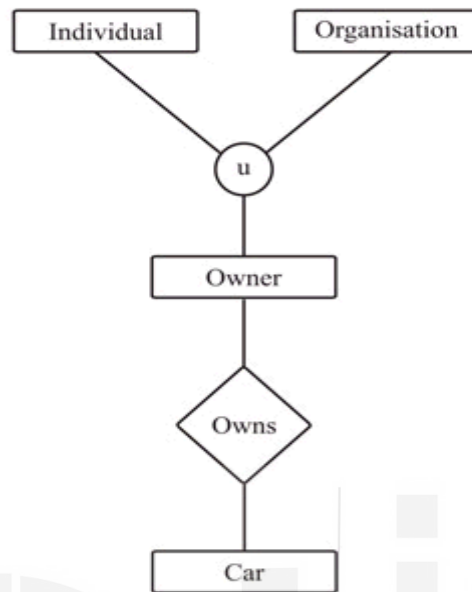


Figure 3.13: Union of classes

In the next section, we discuss how these extended features can be converted to relations.

3.6 CONVERTING EER DIAGRAM TO RELATIONS

The rules for converting the EER diagram, which primarily includes specialisation and generalisation hierarchy are the same as in the E-R diagram. Let us recapitulate these rules:

- Create a relation for each strong entity set.
- Create a relation for a weak entity set. The primary key of this relation would be a composite key involving the attributes of the primary key of the strong entity set on which it depends and discriminator of the weak entity.
- Create a relation for each binary $m : n$ relationship set having the primary keys of both the participating entities, which form the composite primary key to the relation. The individual primary keys are the foreign keys to respective relations.
- For a binary $m : 1$ or $1 : m$ relationship, in general, the primary key on the m side is added to 1 side of the entity. This also becomes the foreign key. For abinary $1:1$ relationship set the primary key of chosen participating relation is added to the other participating relation.
- Composite attributes may sometimes be converted to a separate relation.

- For generalisation or specialisation hierarchy a relation can be created for higher level and each of the lower-level entities. The higher-level entity would have the common attributes and each lower-level relation would have the primary key of the higher-level entity and the attributes defined at the lower specialised level. However, for a complete disjoint hierarchy no relation may be made at the higher level, but the relations are made at the lower level including the attributes of higher level.
- For an aggregation, all the entities and relationships of the aggregation are transformed into the relation on the basis of rules stated above. A relation is also created for the relationship set that exists between the aggregated entity (say AgE) and another simple entity (SE). The primary key of this new relation is the composite key involving the primary key of the AgE and SE.

So let us now discuss the process of converting the EER diagram into a relation. In case of disjoint constraints with total participation. It is advisable to create separate relations for the sub-classes. But the only problem in such a case will be to implement the referential entity constraints suitably.

For example, assuming that this EER diagram can be converted into a relation as:

Car (Number, owner, mileage)
 Scooter (Number, owner, stock)
 Truck (Number, owner, capacity)

Please note that referential integrity constraint in this case would require a relationship with three relations and therefore is more complex to implement.

In case, in the EER diagram of Figure 3.12 there is NO total participation of Vehicle super class in the sub-classes, then there will be some vehicles, which are not Car, Scooter and Truck, so how can you represent these? In addition, in case of overlapping constraints, some tuples may get represented in more than one relation. Thus, in such cases, it is ideal to create one relation for the super class and other relations for the sub-classes having the primary key and any other attributes of that sub-class. For example, with NO total participation the following relations would be created for the EER diagram of Figure 3.12:

Vehicle (Number, owner, type)
 Car (Number, mileage)
 Scooter (Number, stock)
 Truck (Number, capacity)

Finally, in the case of union since it represents dissimilar classes, you may represent separate relations. For example, both individual and organisation will be modeled to separate relations.

Check Your Progress 2

- 1) What is the use of the EER diagram?

.....

.....

.....

2) What are the constraints used in EER diagrams?

.....

.....

.....

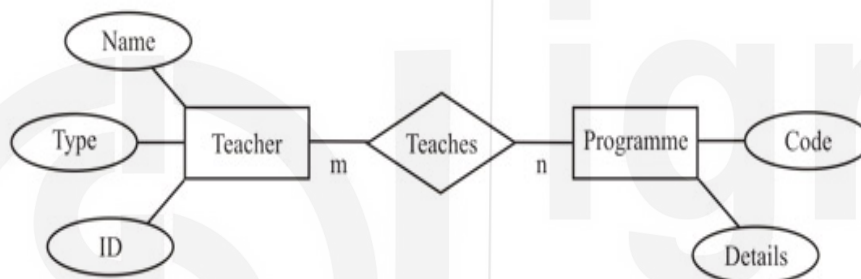
3) How is an EER diagram converted into a relation?

.....

.....

.....

4) Consider the following E-R diagram.



‘Type’ can be regular or visiting faculty. Visiting faculty members can teach only one programme. Make a suitable EER diagram for this and convert the EER diagram to table.

.....

.....

.....

3.7 SUMMARY

This unit presents the concept of E-R model and EER models. Both these models are represented with the help of E-R diagram and EER diagram. These diagrams are very powerful tools to represent the need of data in a database system and can be used for the design of a good database system. The E-R model explained in this unit covers the basic aspects of E-R modeling. The unit defines the concept of entities, attributes and relationships. Further, it defines different types of entities like strong and weak entities; different types of attributes such as simple, composite, derived etc.; the cardinality and participation constraints in a relationship. These concepts are very useful and you should attempt solving related problems from the further readings. Concepts of EER diagrams including generalisation, specialisation, union

etc. have also been explained in this unit. You may refer to further readings for more details on E-R and EER diagrams.

3.8 SOLUTIONS/ ANSWERS

Check Your Progress 1

1.

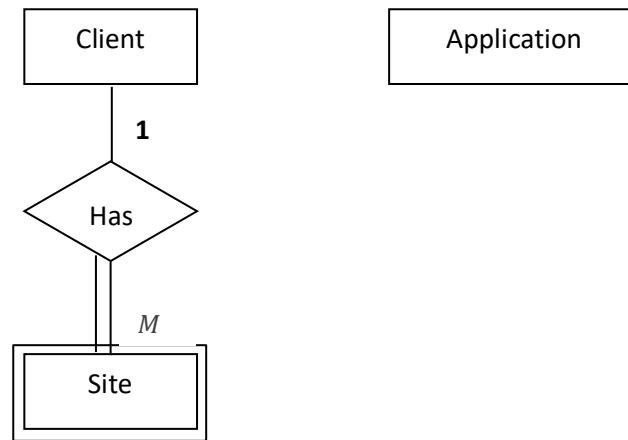
Let us show the step-by-step process of development of Entity-Relationship Diagram for the Client Application system. The first two important entities of the system are **Client** and **Application**. Each of these terms represents a noun, thus, they are eligible to be the entities in the database. But are they the correct entity sets? Client and Application both are **independent** of anything else and the company plans to keep track of its clients and the applications being developed for them. Therefore, each of the entities-Client and Application form an entity set.

But how are these entities related? Are there more entities in the system? Let us first consider the relationship between these two entities, if any. Obviously, the relationship among the entities depends on interpretation of written requirements. Thus, we need to define the terms in more detail.

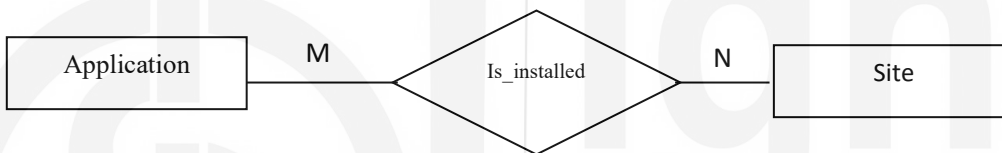
Let us first define the term **Application**. Some of the questions that are to be answered in this regard are: Is keeping track of **Accounts** an application? Is the accounting system installed at each client site regarded as a different application? Can the same application be installed more than once at a particular client site?

Before you answer these questions, do you notice that another entity is in the offering? The client **site** seems to be another candidate entity. This is the kind of thing you need to be sensitive to at this stage of the development of the entity relationship modeling process. So, let us first deal with the relationship between Client and Site before coming back to Application. Just a word of advice: "It is often easier to tackle what seems likely to prove simple before trying to resolve the apparently complex."

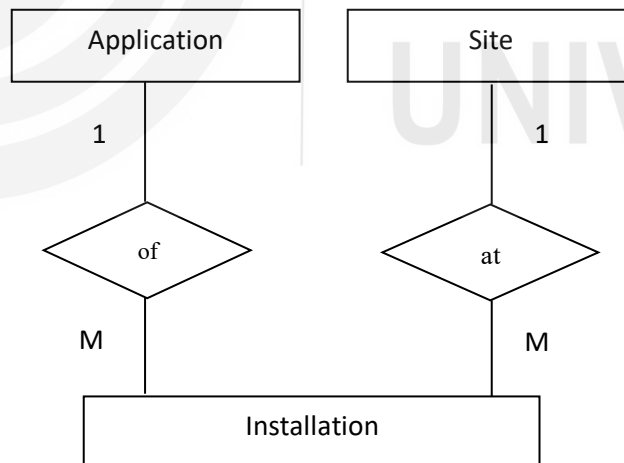
Each Client can have many sites, but each site belongs to one and only one client. Now the question arises what entity type is Site? You cannot have a site without a client. If any site exists without a client, then who would pay the company? This is a good example of an existential dependency and a one-to-many relationship. Thus, Site is a weak entity. This is illustrated in the part E-R diagram given below:



Let us now relate the entity Application to other entities. Please note that several applications developed by the company can be installed at several client sites. Thus, there exists a many-to-many relationship between the entities Site and Application:



However, the M: M relationship “is_installed” has many attributes that need to be stored with it, specifically relating to the authorised persons, setup constraints, dates, etc. Thus, it may be a good idea to promote this relationship as an Entity.



The Figure given above consists of the following relationships:

of - relationship describing the installation of applications and

at - relationship describing the installation at sites.

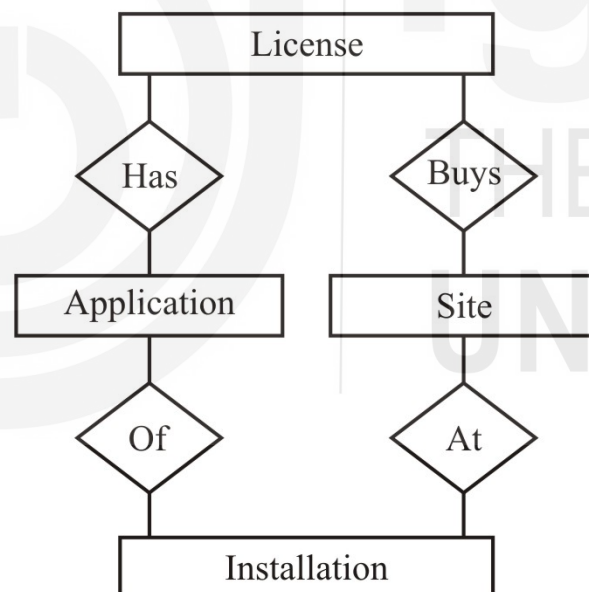
Please note that entities can be recognised in one of two ways – from the nouns of the requirement specification of the system or because of resolving a M:M relationship, as in this case. When you create a new entity in this way, you must find a suitable name for it. This can sometimes be based on the **verbs** used to describe

the M:M. For example, from the statement **you can install the application at many sites**, you can choose the verb install and convert it to related noun **Installation**. But what how will you identify the attributes and relationships of the Installation entity? To find these, you may like to answer the following questions:

- How do you desire to identify each Installation entity?
- What data would be stored in the Installation entity?
- Is an Installation independent of any other entity, that is, can an entity Installation exist without being associated with the entities Client, Site and Application?

In the present design, there cannot be an Installation until you can specify the Client, Site and Application. But since Site is existentially dependent on Client or in other words, Site is subordinate to Client, the Installation can be identified by (Client) Site (it means Client or Site) and Application. You do not want to allow more than one record for the same site and application.

But what if we also sell multiple copies of packages for a site? In such a case, you need to keep track of each individual copy (license number) at each site. In that case, you need another entity named Package (with license number). You may even need separate entities for Application and Package. This will depend on what attributes you want to keep in each of these entities. Thus, with these requirements, the E-R diagram may be extended to as shown below:



Let us define the additional relationships given above:

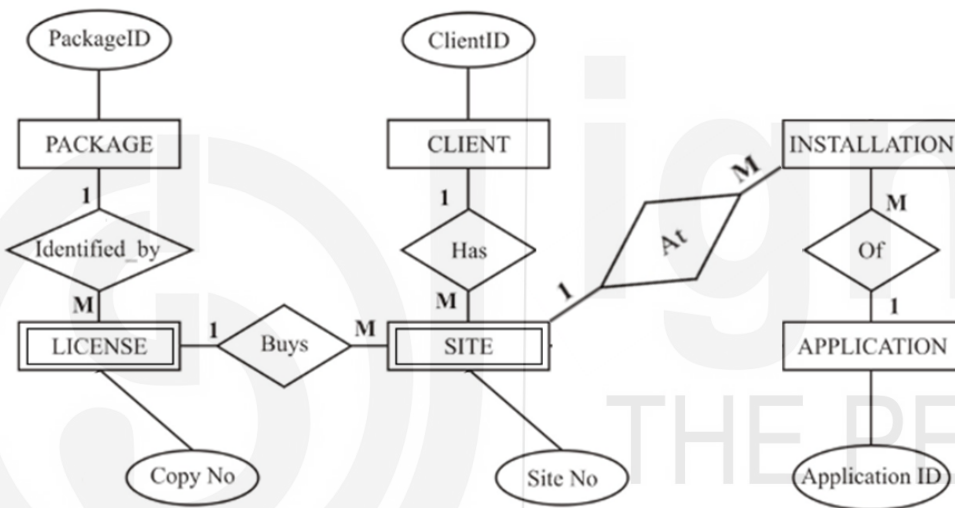
Has: describes that each application has one or more licenses

Buys: describes each site buys the licensed copies of application

You might decide that License should be sub-class to Package entity, therefore, the best unique identifier for the License entity could be Package ID and License serial number. The reason for this relationship is that the license numbers are issued by the companies to whom the Package belongs. The present company does not issue license numbers, and thus have no control over their duration, other service requirements, as

well as the length of data types used for different attributes of the Package and License entity sets. Also, the company does not have control over their uniqueness and changeability of license numbers. Please note that **it is safer to base primary keys on internally assigned values**. What if you make License as a sub-class to Package entity set? It also seems that the client site is not an essential part of the identifier, as the client is free to move the copy of a package to another site. Thus, we should definitely not make client/site part of the primary key of License.

A final proposed E-R diagram for the problem is given below. Please keep thinking and refining your reasoning. Please note that knowing and thinking about a system is essential for making good E-R diagrams. (Please note that in this E-R Diagram, Site and License are modeled as weak entities, though you can decide to change it.)

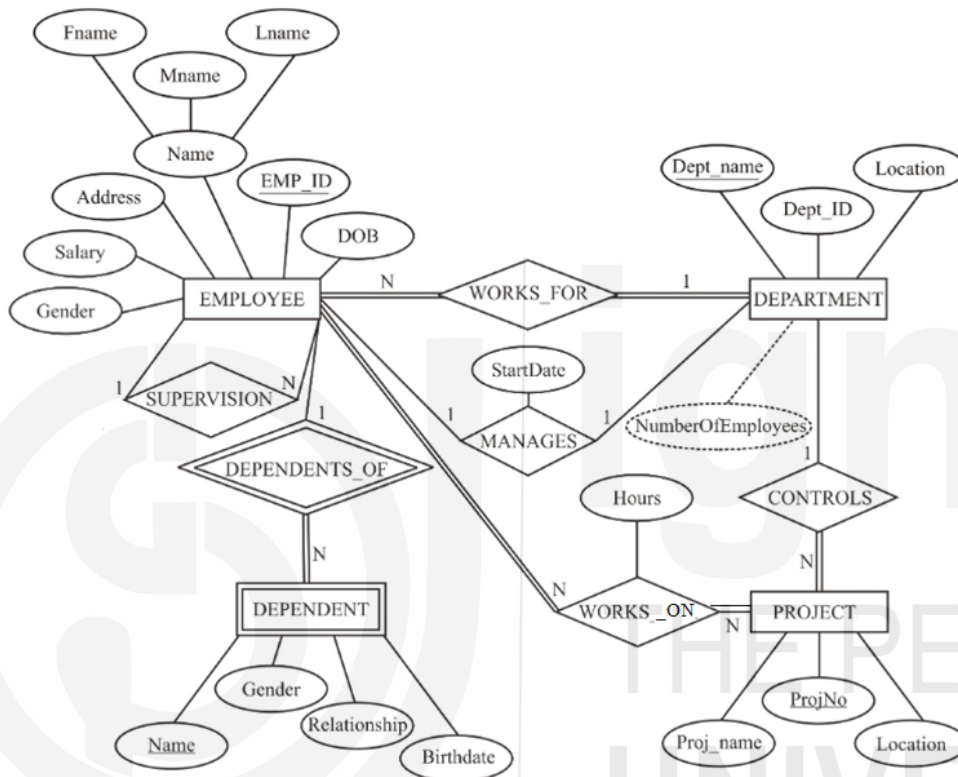


The following table lists probable entities identified so far, together with its superclass, if any, primary keys, and any foreign keys.

Entity	Super Class (if any)	Primary Key	Foreign Keys
Client	-	Client ID	
Site	Client	Client ID, Site No	Client ID
Application	-	Application ID	
Package	-	Package ID	
Installation	Site, Application	Client ID, Site No, Application ID	Client ID, Site No, Application ID

License	Package	Package ID, Copy No	Package ID
---------	---------	---------------------	------------

2) The E-R diagram is given below:



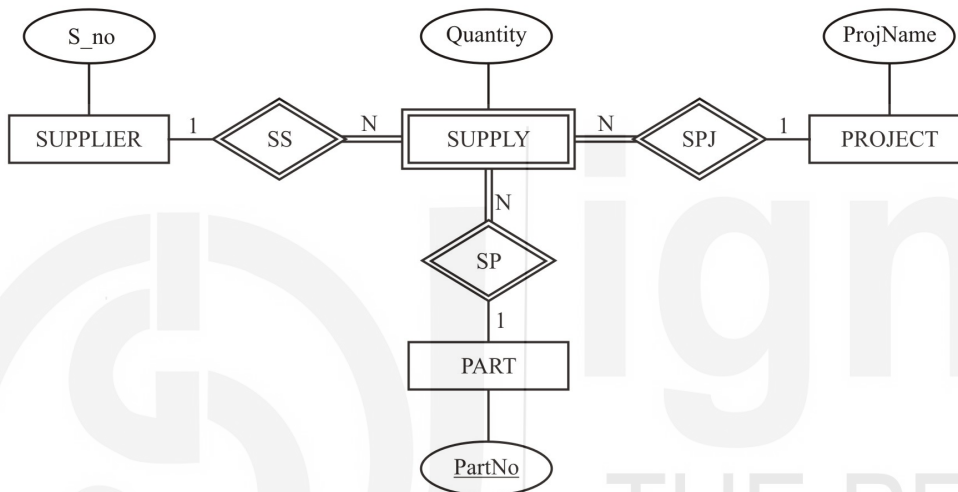
In the E-R diagram given above, EMPLOYEE is an entity, who works for a department, i.e., entity DEPARTMENT, thus, WORKS_FOR is many-to-one relationship, as many employees work for one department. Only one employee (i.e., Manager) manages the department, thus **manages** is the one-to-one relationship. The attribute Emp_Id is the primary key for the entity EMPLOYEE, thus Emp_Id is unique and NOT NULL. The candidate keys for the entity DEPARTMENT are **Dept_name** and **Dept_Id**. Along with other attributes, NumberOfEmployees is the derived attribute on the entity DEPARTMENT, which is the number of employees working for a department. Both the entities EMPLOYEE and DEPARTMENT participate totally in the relationship WORKS_FOR, as at least one employee works for the department, similarly an employee must work in a department.

The entity EMPLOYEES and the entity PROJECTS are related through the many-to-many relationship WORKS_ON, as many employees can work for one or more than one projects simultaneously. The entity DEPARTMENT and entity PROJECT has a relationship CONTROLS. Since one department controls many projects, thus,

CONTROLS in a 1:N relationship. The entity EMPLOYEE participates totally in the relationship WORKS_ON, as each employee works on at least one project. A project should also have at least one employee, therefore, its participation is also total in WORKS_ON.

The employees can have many dependents, but the entity DEPENDENTS cannot exist without the existence of the entity EMPLOYEE, thus, DEPENDENT is a weak entity. You can very well see the primary keys for all the entities. The underlined attributes in the eclipses represent the primary key.

3. The E-R diagram for supplier-and-parts database is given as follows:



4. Let us first make a simple relation for the E-R diagram in the answer to question 2:

EMPLOYEE(EMP_ID, Fname, Mname, Lname, DOB, Address, Salary, Gender)

DEPARTMENT(Dept_ID, Dept_name, Location)

DEPENDENT(EMP_ID, Name, Gender, Relationship, Birthdate)

Foreign Key: EMP_ID references EMPLOYEE

PROJECT(ProjNO, Proj_name, Location)

WORKS_FOR: due to this relationship the Primary key of 1 side (Dept_ID) will be added to the EMPLOYEE relation.

SUPERVISION: this relationship is on the same entity, therefore, an attribute Supervisor_ID whose domain is EMP_ID will be added to the EMPLOYEE

MANAGES: It is a 1 : 1 relation, you can choose the Department side as there will be less records. It also has an attribute StartDate. Therefore,

MANAGER_ID whose domain is EMP_ID and StartDate attribute would be added to DEPARTMENT.

CONTROLS: It is a 1 : N relationship, so the Primary key of 1 side (Dept_ID) will be added to the PROJECT relation.

DEPENDENT_OF: This relation is already included in DEPENDENT relation.

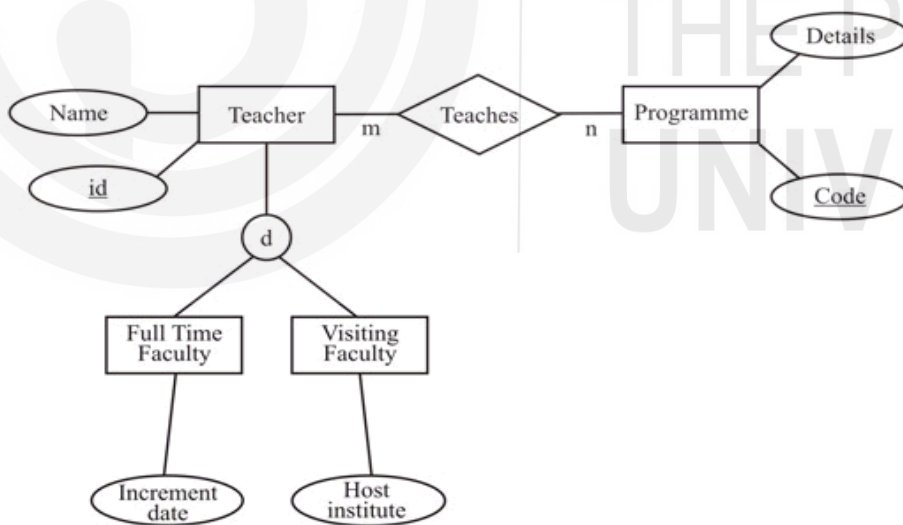
WORKS_ON: It is a many to many (N : N) relation with total participation on both sides, therefore, a separate table will be created for WORKS_ON with primary key of both the participating entities and attributes of this relation (Hours)

Thus, the final relations would be:

EMPLOYEE (EMP_ID, Fname, Mname, Lname, DOB, Address, Salary, Gender, Dept_ID, Supervisor_ID)
 Foreign Key: Dept_ID references DEPARTMENT.
 Domain Constraint: Domain of Supervisor_ID is EMP_ID.
 DEPARTMENT (Dept_ID, Dept_name, Location, MANAGER_ID, StartDate)
 Foreign Key: MANAGER_ID references EMP_ID of EMPLOYEE.
 DEPENDENT (EMP_ID, Name, Gender, Relationship, Birthdate)
 Foreign Key: EMP_ID references EMPLOYEE
 PROJECT (ProjNO, Proj_name, Location, Dept_ID)
 Foreign Key: Dept_ID references DEPARTMENT.
 WORKS_ON (EMP_ID, ProjNo, Hours)

Check Your Progress 2

- 1) The EER diagrams are used to model advanced data models requiring inheritance, specialisation and generalisation.
- 2) The basic constraints used in EER diagrams are disjointness, overlapping and unions.
- 3) For disjointness and union constraints the chances are that you create separate relations for the subclasses and no relation for super class. For overlapping constraints, it is advisable to create a relation of super class and the relations of sub-classes will have only those attributes that are not common to super class except for the primary key.
- 4) The modified EER diagram is:



FullTimeFaculty (id, Name, Increment-date)

VisitingFaculty (id, Name, Host-institute)

Teachers (id, code)

Programme (code, details)